Tianshu 插件开发者手册

xiaosi 插件库　*TPI 1.0*　*XPI v1*

01 开发概览

TPI（Tianshu Plugin Interface）定义插件如何进入天枢的会话、注册工具和使用宿主能力。XPI 是 xiaosi 插件库的发行协议，负责发现版本、发布审核、安装授权和签名校验。按快速开始创建插件并在天枢验证，再通过开发者中心或 CI 提交精确版本。

阅读目录

我想做什么	阅读
创建并试运行第一个文本统计工具	02 快速开始
声明插件 ID、入口、工具、权限和宿主版本	03 清单与兼容性
实现工具，处理会话数据、生命周期与取消	04 TPI SDK 与运行时
打包、选择分区、申请审核或撤回版本	05 打包、发布与更新
从构建流水线校验、提交并跟踪版本	06 开发者 API 与 CI
集成宿主设备证明和固定云端巡检	07 设备证明与云端巡检

下载与接口契约

- TPI 1.0 类型声明位于 `tpi/tpi.d.mts`。将它与插件的 `tpi.mjs` 放在同一目录，供编辑器和 TypeScript 使用。
- 开发者中心（`/developers/authorization`），管理插件 ID、发行版本、付费测试资格和 CI 凭据。
- 能力发现（`/api/v1/tianshu/capabilities`）与 OpenAPI 3.1（`/api/v1/openapi.json`），查询服务当前开放的能力并生成接口客户端。
- 插件清单（`/api/v1/tianshu/schemas/manifest`）、发行记录（`/api/v1/tianshu/schemas/release`）和内容摘要表（`/api/v1/tianshu/schemas/contents`）的 JSON Schema。

手册中的站点路径以 `/` 开头，请拼接到 XPI 服务的可信站点源。命令中的 `https://xiaosiai.cn` 为示例，联调时使用实际部署的地址。

目录 `docs/schemas/` 收录三份 JSON Schema，`docs/openapi.json` 收录面向插件和宿主开发的 HTTP 契约。在线文件见公开接口仓库。

首期支持范围

插件库接收 TPI 1.0 纯 JavaScript agent 发行包，使用 `stable` 通道、数据模式 `1` 和迁移类型 `none`。在真实天枢上测试后再声明平台。XPI 安装与云端工具还要求支持对应功能的天枢宿主、可信引导文件和有效设备授权。付费接入使用测试结账，当前宿主阻止付费未审核包执行。已开通的云端功能是针对运营方登记目标的 HTTPS 巡检。

02 快速开始

示例插件注册 `count_text` 工具，统计输入中的 Unicode 码点。准备支持 TPI 1.0 的天枢测试宿主。示例清单声明最低核心版本为天枢 `4.0.2`。手动打包步骤使用 Node.js 和 ZIP 工具。

第 1 步 创建插件目录

新建目录 `my-plugin`。将第 3 章的最小清单保存为 `tianshu.plugin.json`，再按第 4 章的代码创建 `tpi.mjs` 和 `index.mjs`。发布时把清单和入口的 `my-plugin` 改为自己登记的插件 ID。工具名称须与清单一致。

第 2 步 在天枢中试运行

在测试环境的天枢终端界面输入 `/plugin install /绝对路径/my-plugin`。核对天枢展示的清单和权限，再按终端提示加上 `--confirm` 完成安装。在新会话中用 `count_text` 检查一段非敏感文本。修改插件后，重新安装测试目录并启动新的宿主进程。

将 `{"text":"甲😀\r\n乙\n"}` 传给工具，应返回字符串 `"6"`。

本地安装从插件目录加载 JavaScript。签名包的安装、更新和授权可在支持 XPI 的天枢中使用测试插件库验证。操作步骤见实例的安装文档（`/docs#install`）。

第 3 步 打包与验包

在开发者中心（`/developers/authorization`）完成发布者核验并登记插件 ID。登录后从发布者资料（`/api/v1/developer/profile`）取得 `publisher_id`。按第 5 章生成发行 ZIP，再用第 6 章的 HTTP 接口验包。

第 4 步 提交版本

登录开发者中心（`/developers/authorization`），提交刚验证的 ZIP、软件许可、数据用途、HTTPS 源码地址和版本说明。按打包、发布与更新选择分区并跟踪结果。需要在流水线里提交时，按开发者 API 与 CI 为当前插件和分区签发 CI 凭据。

已有离线开发套件

已经持有离线套件 ZIP 的开发者可继续使用原命令。套件需要 Node.js `>=24.22.1 <27` 和 npm。将命令中的 ZIP 文件名替换为手头的文件名。

```
mkdir tianshu-devkit
unzip tianshu-developer-kit.zip -d tianshu-devkit
cd tianshu-devkit
npm ci
npm run plugin -- init ./my-plugin --id my-plugin
npm test
npm test -- ./my-plugin
```

套件自带的 `count_text` 同时统计码点与文本行。上例在套件版本中的预期结果是 `{"codePoints":6,"lines":3}`。修改工具后，按自己的输入输出调整本地测试。

完成发布者登记后，在套件目录运行以下命令。使用自己的发布者 ID、源码 Git 提交、唯一发行 ID 和已验证的平台。

```
npm run plugin -- pack ./my-plugin ./my-plugin.zip \
  --release-id rel_my_plugin_100 \
  --publisher-id publisher-id \
  --source-commit 0123456789abcdef0123456789abcdef01234567 \
  --platforms darwin-arm64,linux-x64
npm run plugin -- validate ./my-plugin.zip --lane official_reviewed
```

03 清单与兼容性

插件根目录必须包含 UTF-8 文件 `tianshu.plugin.json`。宿主读取此清单并校验入口握手。插件库同时验证清单、发行记录和内容摘要表。本章描述插件库首期可发行的范围。清单 JSON Schema（`/api/v1/tianshu/schemas/manifest`）

最小工具插件清单

```
{
  "$tpi": "1.0",
  "kind": "tianshu.plugin",
  "id": "my-plugin",
  "displayName": "文本统计",
  "version": "1.0.0",
  "description": "统计输入文本的 Unicode 码点",
  "runtime": "agent",
  "entry": "index.mjs",
  "capabilities": ["tools"],
  "contributes": {
    "tools": [{ "name": "count_text", "description": "统计文本" }],
    "lifecycle": []
  },
  "permissions": {},
  "engines": { "tianshu": ">=4.0.2 <5.0.0" }
}
```

字段	发行约束
`$tpi`、`kind`、`runtime`	分别为 `1.0`、`tianshu.plugin`、`agent`，运行形态省略时为 `agent`。
`id`	1 至 128 个字符，以字母或数字开头，后续允许字母、数字、点、下划线与连字符。尾点、系统保留文件名和内置插件 ID `xiaosi-axiom` 禁用。与入口定义和发行记录一致。
`displayName`、`description`	均必填，分别最多 64 与 500 个字符，可用中文。
`version`	最多 32 个字符的稳定 SemVer，例如 `1.2.3`，仅接受主、次、修订版本。
`entry`	包内相对路径，指向非空 UTF-8 `.js`、`.mjs` 或 `.cjs` 文件。推荐使用 `.mjs` 明确 ESM 模块格式。
`capabilities`	列出实际使用的宿主能力。贡献工具时须含 `tools`，详见 TPI SDK 与运行时。
`contributes.tools`	工具名称与入口提供的工具一一对应。名称最多 64 个字符，匹配 `^[a-z][a-z0-9_]*$`，插件内唯一。描述最多 200 个字符。

字段	发行约束
`contributes.lifecycle`	支持 `turnStart`、`turnEnd`、`toolEnd` 和 `sessionEnd`。agent 至少贡献一个工具或一个生命周期阶段。
`contributes.skills`	每项为包内相对目录，必须含 `SKILL.md`。
`contributes.commands`	包内 Markdown 文件路径，或包含 Markdown 文件的目录。
`contributes.hookScripts`	每项包含 `event`、包内 JavaScript `script` 以及可选的正整数 `timeoutMs`。详见 TPI SDK 与运行时。
`permissions`	`fs`、`net`、`shell` 各为可选布尔值，供审查和安装提示。只声明真实用途。
`engines.tianshu`	可选 SemVer 范围。如果填写，归一化后须与发行记录的 `compatibility.core` 一致。
`supersedes`、`builtin`	前者为启用插件时让位的内置工具名称，可省略。后者省略或填 `false`。

清单不接受额外字段。插件库不收桌面托管插件。付费未审核分区要求至少一个工具，禁止声明生命周期和外部 hook。

发布前验证兼容性

`xiaosi.release.json` 的 `compatibility` 声明核心 SemVer 范围 `core`、协议 `tpi: "1.0"` 和实际验证过的 `platforms`。平台用 Node.js 的 OS 和架构名称拼接，支持 `darwin`、`linux`、`win32` 与 `x64`、`arm64` 的组合。填写平台前至少验证入口加载、工具调用和插件使用的宿主服务。

公开 兼容性 API 按请求的准入分区查询已公开版本，并比较核心版本、TPI 版本和平台。兼容结果只覆盖声明中的条件。真实安装还会检查签名目标、当前资格、设备授权及安装确认。可选能力是否可用依然取决于会话配置。

04 TPI SDK 与运行时

入口和会话

宿主先安装只读的 `globalThis.__tianshuTPI`，随后加载插件入口。保存下面的模块为 `tpi.mjs`，并可在同一目录放入 `tpi.d.mts` 类型声明。

```
const sdk = globalThis.__tianshuTPI;
if (typeof sdk?.defineTianshuPlugin !== "function") {
  throw new Error("请在支持 TPI 1.0 的天枢中加载插件。");
}
export const { defineTianshuPlugin } = sdk;
```

在 `index.mjs` 中默认导出或以具名 `plugin` 导出 `defineTianshuPlugin({ id, activate })` 的返回值。把 `id` 设为清单 ID，把会话资源的初始化放入 `activate(host)`。

```
import { defineTianshuPlugin } from "./tpi.mjs";

export default defineTianshuPlugin({
  id: "my-plugin",
  activate(host) {
    host.log.debug("文本统计工具已注册");
    return {
      tools: [{
        name: "count_text",
        description: "统计文本中的 Unicode 码点",
        inputSchema: {
          type: "object",
          properties: { text: { type: "string", maxLength: 10000 } },
          required: ["text"],
          additionalProperties: false
        },
        concurrencySafe: true,
        async execute({ args }) {
          if (typeof args.text !== "string") {
            return { content: "请输入文本。", isError: true };
          }
          const codePoints = Array.from(args.text).length;
          if (codePoints > 10000) {
            return { content: "请输入最多 10000 个 Unicode 码点。", isError: true };
          }
          return { content: String(codePoints) };
        }
      }]
    };
  }
});
```

会话绑定时宿主检查工具名与清单是否一致。返回未声明的生命周期处理器或与已有工具重名时，会话会拒绝接入插件。需要关闭文件、数据库或定时器时返回 `dispose()`，清理当前会话持有的资源。

工具接口

- `execute({ args, cwd, sessionId, signal })` 接收展平的模型参数，返回 `{ content: string, isError?: boolean }`。按真实输入验证类型、长度和范围，使用 `cwd` 作为会话工作目录。
- 对接收文件路径的工具，为相应参数声明 `pathParams: { file_path: "read" }` 或 `"write"`。宿主会对声明的非空字符串调用路径守卫，校验通过后传入规范化的绝对路径。工具仍须检查参数类型和预期文件范围。
- `requiresApproval: true` 标记需要用户审批的工具。仅对只读且无副作用的操作设置 `concurrencySafe: true`。使用 `timeoutMs` 设置宿主管理的调用预算，同时为自己的外部调用设置超时。
- SDK 的工具调用类型含可选 `signal`。当前基线的工具包装器未转发该字段。代码可在收到信号时协作退出，并应为无信号的调用自行设置超时和清理。宿主超时无法终止已在同进程运行的任意代码。

宿主能力

`host` 含 `tpiVersion`、`pluginId`、`cwd`、可选 `sessionId` 和 `isWorker`。下表的可选服务取决于清单和会话配置。

能力	接口与边界
`tools`	通过 `activate` 返回工具数组。
`storage`	`host.storage?.dataDir()` 返回插件数据目录，`openDatabase("state.db")` 打开该目录下的 SQLite 数据库。使用单一相对文件名，及时关闭连接。
`account`	`host.account?.userKey()` 返回账号 ID 的稳定哈希，未登录时为 `local-default`。
`embeddings`	`available()`、`modelId()`、`embed(texts, signal?)`。通道由宿主配置，可能不可用或使用远程服务。记录模型标识以判断何时重建本地向量。
`sidePathLLM`	`available()`、`complete(prompt, { timeoutMs, signal? })`。仅在宿主装配该会话通道时调用。
`promptContext`	`setBlock(blockOrNull)` 仅在 `turnStart` 生效。块使用单一 XML 根元素，根标签以标准化插件 ID 为前缀。传 `null` 清除当前块。
`transcripts`	`listSessions(limit?)`、`read(sessionId)`，只读当前项目历史会话的用户与助手文本。
`log`	宿主提供 `host.log.debug(message)` 和 `warn(message)`。日志内容应省略令牌、原始会话文本和敏感参数。

天枢以进程内 JavaScript 运行 agent 插件。清单 `permissions.fs/net/shell` 是用户可见的用途声明。能力清单约束宿主交给插件的接口，文件、网络、进程与环境访问仍受天枢进程自身权限支配。避免用 Node.js 直接访问插件范围外的数据。私有目录是存储约定，隔离取决于宿主环境。声明会读取哪些数据、是否持久化、是否传往模型或外部服务、保留多久及删除方式。`isWorker` 为真时，跳过用户数据写入，防止子会话重复写入。

生命周期与外部 hook

SDK 阶段	事件与执行
`turnStart`	收到用户输入、轮次模式与轮次编号，具有可选 `signal`。基线预算为 2 秒。
`turnEnd`	收到轮次编号、用户输入和助手最终文本，具有可选 `signal`。
`toolEnd`	收到轮次编号、工具名、成功状态和可选目标及结果预览。预览最多 400 个字符。
`sessionEnd`	后台处理，基线预算 180 秒，具有可选 `signal`。

先在 `contributes.lifecycle` 声明阶段，再在 `activate` 返回相应处理器。开始异步任务前检查可用的中止信号，将其传给支持取消的操作，在 `finally` 释放资源。避免把关键的持久化工作全部留到后台会话结束。

contributes.hookScripts 是独立的观察机制，事件名为 preTurn、postTurn、postTool、postSession、onError。插件库要求脚本是发行包内 JavaScript 文件。外部 hook 与 SDK 生命周期分别声明，适合有限的观察工作。付费未审核包禁用这两类自动入口。

测试

对自己的工具测试空值、边界和无效数据，对网络与存储测试缺席、超时和失败清理。在计划支持的天枢版本及操作系统上测试加载、路径守卫、审批、会话关闭与权限声明。提交前使用第 6 章的验包接口检查静态产物，并在宿主中确认实际行为。

05 打包发布与更新

发行 ZIP

发行 ZIP 的根目录直接包含插件入口、资源和以下三个 JSON 文件。

根文件	用途
tianshu.plugin.json	TPI 1.0 插件清单。
xiaosi.release.json	schema: "xiaosi.release/1"，绑定发行 ID、插件 ID、发布者 ID、稳定版本、源码提交、权限摘要、兼容性、数据模式和迁移类型。
xiaosi.contents.json	schema: "xiaosi.contents/1"，files 记录其他所有文件相对路径及小写 SHA-256。

发行记录中 plugin_id 和 version 与清单一致，source_commit 是 40 或 64 位小写 Git 提交摘要，permission_digest 是清单权限对象（缺省 {}）的 JCS 规范化 JSON SHA-256。当前固定 runtime: "agent"、channel: "stable"、data_schema: 1、migration_kind: "none"。使用 发行记录 Schema（/api/v1/tianshu/schemas/release） 和内容表 Schema（/api/v1/tianshu/schemas/contents）检查字段。

约束	上限或规则
ZIP	最多 20 MiB 压缩体积、64 MiB 解包体积、2,048 项，元数据 JSON 每份最多 1 MiB。
压缩格式	ZIP32，支持存储和 Deflate。单项解包大小不超过 1 KiB 与压缩体积 200 倍中的较大者。
文件路径	使用 / 分隔的相对路径。最多 512 字符、32 层、每段 128 字符。避免绝对路径、点段、重名、大小写碰撞、符号链接和系统保留名。
JavaScript	入口及声明的 hook 必须在包内。禁止依赖目录、运行时安装、包脚本、锁文件、原生二进制、WASM 和原生构建脚本。
包内容	只纳入运行所需的代码、许可和资源。排除凭据、.env、.npmrc、.git、测试输出和一次性文件。

在仓库中保留对应源码版本和最终 ZIP 的 SHA-256，CI 和人工审核均提交同一份 ZIP。发布记录以插件 ID 和版本唯一识别。包内容、清单、许可、用途或发布说明需要修改时，使用新 SemVer 和新发行

ID。按破坏性变更、向后兼容功能和修复分别递增主、次、修订号。重复提交相同 ZIP、分区和元数据可以返回已存在的记录。

手动打包

以第 2 章创建的插件为例。在插件目录创建 `xiaosi.release.json`，将示例中的发行 ID、插件 ID、版本、发布者 ID、平台和源码 Git 提交替换为实际值。示例权限摘要对应空权限对象 `{}`。权限变更时按 RFC 8785 规范化 `permissions`，计算其 UTF-8 字节的 SHA-256。

```
{
  "schema": "xiaosi.release/1",
  "release_id": "rel_my_plugin_100",
  "publisher_id": "publisher-id",
  "plugin_id": "my-plugin",
  "version": "1.0.0",
  "channel": "stable",
  "runtime": "agent",
  "compatibility": {
    "tpi": "1.0",
    "core": ">=4.0.2 <5.0.0",
    "platforms": ["darwin-arm64"]
  },
  "data_schema": 1,
  "migration_kind": "none",
  "permission_digest":
"44136fa355b3678a1146ad16f7e8649e94fb4fc21fe77e8310c060f61caaff8a",
  "source_commit": "0123456789abcdef0123456789abcdef01234567"
}
```

在同一目录运行下列命令，按原始文件字节生成内容摘要表，并将文件添加到新 ZIP。额外的资源文件应同时加入摘要和 ZIP 的文件列表。修改任何包内文件后重新生成摘要与 ZIP。

```
node --input-type=module - >xiaosi.contents.json <<'JS'
import { createHash } from "node:crypto";
import { readFileSync } from "node:fs";
const paths = ["tianshu.plugin.json", "index.mjs", "tpi.mjs", "xiaosi.release.json"];
const files = Object.fromEntries(paths.map(path => [
  path, createHash("sha256").update(readFileSync(path)).digest("hex")
]));
console.log(JSON.stringify({ schema: "xiaosi.contents/1", files }, null, 2));
JS

test ! -e ../my-plugin.zip && zip -X -0 ../my-plugin.zip \
  tianshu.plugin.json index.mjs tpi.mjs xiaosi.release.json xiaosi.contents.json
```

将新 ZIP 交给第 6 章的验包 API，核对 `valid`、发行 ID 和摘要。已有离线套件也可使用第 2 章的原打包命令。

选择准入分区

分区	流程	安装与更新
免费官方授权 `official_reviewed`	核验发布者，登记插件 ID，提交 ZIP 与披露信息，申请官方审核。审核绑定到该版本的精确 ZIP 摘要，通过后由管理员签名发布。	用户确认安装。满足宿主条件且用户开启时，合格更新可自动应用。
付费授权 `paid_unreviewed`	发布者提交同类资料，版本须通过技术检测并取得该插件的有效接入资格。现有结账走测试沙盒，目录明确显示未经官方审核。	每个版本都要求额外确认，禁止自动更新。当前宿主阻止此分区的插件执行。

在开发者中心（`/developers/authorization`）提交时提供许可标识、数据用途、HTTPS 源码地址和更新说明。数据用途说明应覆盖本地读写、网络或模型调用、发送的数据、保留与删除方式。对使用者收取的外部服务费用也应披露。付费接入费由发布者承担，个人激活由设备授权流程管理。

跟踪、更新与撤回

- 在开发者中心查看技术状态、版本状态、审核结论、发行摘要与当前策略修订号。审核补充资料按反馈处理。任何改变包字节的修改都需要新版本。
- 自动更新只在官方区有资格。除递增版本外，还须满足当前授权及宿主兼容性，权限、能力、贡献项、数据用途及模式保持一致，迁移类型为 `none`。否则需要重新核对并确认。更新在宿主重启边界切换。
- 撤回使用版本所有者的浏览器会话、当前密码、撤回原因和最新策略修订号。更新后重新查询状态。其他操作改变修订号时，先读取最新记录。撤回通过平台资格与宿主重新验证生效。已签离线权益有有效期，离线设备无法立即获得新状态。

06 开发者 API 与 CI

接口统一以站点可信源的 `/api/v1` 为前缀。OpenAPI 3.1（`/api/v1/openapi.json`） 描述路径、请求体和响应。认证失败或请求错误返回 `{code,message,request_id,retryable}`。日志可保留请求 ID、错误码与版本号，避免记录凭据、密码和用户数据。

能力发现与兼容性

这些接口可公开调用。

方法与路径（相对 `/api/v1`）	用途
`GET /tianshu/capabilities`	查协议、发行限制、启用的服务和契约链接。
`GET /tianshu/schemas/:name`	下载 `manifest`、`release` 或 `contents` 的 JSON Schema。
`GET /plugins/:id/releases/:version`	查指定分区公开的精确版本，返回 `{release}`。查询参数 `admission_class` 默认 `official_reviewed`。
`POST /tianshu/compatibility`	比较已发布版本与指定宿主。省略 `version` 时优先返回分区中最新的兼容公开版本，全部不兼容时返回最新版本及原因。

```
export XPI_ORIGIN='https://xiaosiai.cn'
curl -fsS "$XPI_ORIGIN/api/v1/tianshu/capabilities"

curl -fsS "$XPI_ORIGIN/api/v1/tianshu/compatibility" \
  -H 'Content-Type: application/json' \
  --data '{
    "plugin_id": "my-plugin",
    "version": "1.0.0",
    "admission_class": "official_reviewed",
    "host": {"core":"4.0.2","tpi":"1.0","os":"darwin","arch":"arm64"}
  }'
```

响应 `status` 为 `compatible`、`incompatible` 或 `unavailable`。检查 `reasons` 中的 `release_unavailable`、`tpi_version`、`core_version` 和 `platform`。版本不可用时，`version` 和 `release_id` 为 `null`。未公开的开发版本可在提交前用验包接口检查声明。

身份与令牌

凭据	使用场景
浏览器会话	站点用 HttpOnly Cookie 维持登录。登录后的写请求需要同源 `Origin` 和会话响应提供的 `X-XPI-CSRF`。签发、轮换或吊销 CI 凭据，以及登记插件与撤回版本需要当前密码。
CI Bearer `XSPCI1_…`	绑定一个发布者、一个插件 ID、一条准入分区和明确授权项，有效期最长 30 天。
付费发布者凭据 `XSPUB1_…`	用于其付费接入资格绑定的原有提交路径。
XPI 设备证明	用于宿主激活、安装、更新和云端巡检。详见设备证明与云端巡检。

在开发者中心（`/developers/authorization`）使用已核验的发布者账号登记插件 ID，再创建 CI 凭据。同一插件和分区最多保留 5 个有效 CI 凭据。凭据原文只在签发或轮换时返回一次，保存在流水线机密变量 `XPI_PUBLISHER_TOKEN` 中。轮换会吊销原凭据，完成后同步替换流水线的密钥。泄露时立即吊销。避免在命令跟踪、构建输出、Git 仓库或 ZIP 中写入令牌。CI 只需以下与工作相关的权限。

Scope	接口
`packages:validate`	`POST /developer/packages/validate` 接收原始 ZIP 和查询参数 `admission_class`，返回 `valid`、发行与插件 ID、版本、产物摘要、清单摘要、体积与兼容性。
`releases:submit`	`POST /submissions` 接收原始 ZIP 与分区、许可、数据用途、HTTPS 源码 URL 和发行说明。首次成功为 HTTP 201，同包与相同元数据的重试为 HTTP 200。
`releases:read`	`GET /developer/plugins`、`GET /developer/plugins/:id/releases`、`GET /developer/releases/:id`。列表带 `cursor` 和 `limit`，详情返回 `{submission,reviews,review_request}`。
`reviews:request`	`POST /developer/releases/:id/review-request`，携带精确包 SHA-256、当前修订号和备注。

`GET /developer/profile` 返回 `publisher_id`、核验状态、凭据类型、插件、准入分区与 scopes。浏览器资料中的插件和分区为 `null`。CI 的记录范围按提交时选定的 `requested_class` 隔离。CI 无法付款、执行官方审批、签名、撤回或使用宿主设备权限。

CI 示例

先按第 2 章生成 ZIP。沿用上文的 `XPI_ORIGIN`，或将其设为目标环境的 HTTPS 源，省略末尾斜线和路径。由流水线的机密变量设置 `XPI_PUBLISHER_TOKEN`。以下示例使用官方分区的凭据和 curl 7.87.0 或更新版本。

验包以 `Content-Type: application/zip` 上传原始文件字节。

```
curl -fsS "$XPI_ORIGIN/api/v1/developer/profile" \
  -H "Authorization: Bearer $XPI_PUBLISHER_TOKEN"
curl -fsS \
  "$XPI_ORIGIN/api/v1/developer/packages/validate?admission_class=official_reviewed" \
  -H "Authorization: Bearer $XPI_PUBLISHER_TOKEN" \
  -H 'Content-Type: application/zip' --data-binary @./my-plugin.zip
```

验包通过后可提交同一份 ZIP。将示例中的许可、用途、源码 URL 和发行说明替换为实际值。

```
curl -fsS "$XPI_ORIGIN/api/v1/submissions" \
  -H "Authorization: Bearer $XPI_PUBLISHER_TOKEN" \
  -H 'Content-Type: application/zip' \
  --url-query 'admission_class=official_reviewed' \
  --url-query 'license=MIT' \
  --url-query 'data_use=仅在本机统计输入文本，不持久化或传出。' \
  --url-query 'source_url=https://example.com/my-plugin' \
  --url-query 'release_notes=首个稳定版本' \
  --data-binary @./my-plugin.zip

curl -fsS "$XPI_ORIGIN/api/v1/developer/releases/rel_my_plugin_100" \
  -H "Authorization: Bearer $XPI_PUBLISHER_TOKEN"
```

提交成功后，读取精确版本详情，将其中 `artifact_sha256` 与 `policy_revision` 分别填入官方审核申请的 `artifact_sha256` 和 `expected_revision`，并添加 `note`。审核申请进入 `queued`，审核员决定后进入 `decided`，撤回版本时待审申请进入 `cancelled`。平台另行发布与签名。

```
curl -fsS \
  "$XPI_ORIGIN/api/v1/developer/releases/rel_my_plugin_100/review-request" \
  -H "Authorization: Bearer $XPI_PUBLISHER_TOKEN" \
  -H 'Content-Type: application/json' --data @- <<'JSON'
{
  "artifact_sha256":
    "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef",
  "expected_revision": 1,
  "note": "已在指定天枢和平台测试。"
}
JSON
```

已有离线开发套件

已有离线套件的开发者可在套件目录使用以下命令，使用相同的可信站点源、CI 凭据和发行元数据。

```
npm run plugin -- validate ./my-plugin.zip --lane official_reviewed --remote
npm run plugin -- submit ./my-plugin.zip --lane official_reviewed \
  --source-url https://example.com/my-plugin --license MIT \
  --data-use '仅在本机统计输入文本，不持久化或传出。' \
  --release-notes '首个稳定版本'
npm run plugin -- status rel_my_plugin_100
```

发布者浏览器接口

方法与路径（相对 `/api/v1`）	主要字段
`POST /developer/plugins`	`plugin_id`、`password`
`POST /developer/releases/:id/withdraw`	`reason`、`expected_revision`、`password`
`GET /developer/keys`	返回 `{keys}`，含令牌 ID、范围、提示、创建、过期与吊销时间。
`POST /developer/keys`	`plugin_id`、`admission_class`、`scopes`、`expires_in_days`、`password`
`POST /developer/keys/:id/rotate`	`expires_in_days`、`password`
`DELETE /developer/keys/:id`	`password`，成功返回 HTTP 204。

错误和重试

HTTP 或错误	处理
400 `INVALID_REQUEST`、413 或 415	检查请求字段、查询参数、包大小与 MIME。
401 `AUTH_INVALID` 或 403	检查凭据的签发环境、有效期、发布者核验、插件归属、准入分区与 scope。浏览器同时核对 Origin 和 CSRF。
404 `NOT_FOUND`	核对分区、发行 ID、版本可见性与记录归属。
409 `STATE_CONFLICT`	重新读取版本和策略修订号。若现有版本的 ZIP 或提交元数据不同，使用新版本和发行 ID。
409 `KEY_LIMIT` 或 `RELEASE_LIMIT`	检查该插件和分区的有效 CI 凭据数量，或联系平台维护者处理发行容量。
422 `ARTIFACT_REJECTED`	检查清单、路径、内容摘要及分区限制，再调用验包接口。
429 `RATE_LIMITED` 或可重试 5xx	对 `retryable: true` 使用有上限的退避。有 `Retry-After` 时按其等待。提交重试应保留原始 ZIP 和元数据。

07 设备证明与云端巡检

宿主信任边界

使用安全渠道交付的 `trust.json` 初始化宿主，其中包含可信站点 `origin`、TUF 初始根与权益签名公钥。开发和生产环境使用独立的可信引导与密钥。首次建立信任时，通过运营方给出的独立渠道核对该文件。网络返回的根只用于受信任 TUF 更新链。

宿主以精确版本生成安装计划，核对 TUF 目标、包大小、ZIP 与清单 SHA-256、权限、数据用途和平台资格。在执行安装或更新前按计划确认。权益 JWS 使用固定 EdDSA 算法，校验发行者、受众、签名键、设备密钥指纹、插件范围、准入分区、时间与修订号。签发的离线权益最长 7 天。访问、刷新和下载凭据均由宿主保存，插件及发布流水线应只持有各自所需权限。对用户展示计划和授权来源。详见使用文档（`/docs#install`）与 OpenAPI（`/api/v1/openapi.json`）。

设备请求签名

宿主为设备生成 Ed25519 密钥。设备私钥留在宿主的凭据存储。以下协议供原生宿主集成使用。已集成 XPI 的客户端应使用其原生签名通道。

1. 向 `POST /api/v1/auth/challenges` 提交 JSON `{purpose,public_key}`。公钥是 32 字节裸 Ed25519 公钥的 base64url 编码。purpose 为 `activate`、`refresh` 或 `device`。
2. 验证返回的 `audience` 等于信任文件的 `origin + "/api/v1"`。挑战 90 秒有效且只能消费一次。每次重新发起请求使用新挑战。
3. 将下列数组按 `JSON.stringify` 序列化为 UTF-8 并签署。方法大写，请求路径包括原始查询字符串。JCS 使用 RFC 8785，GET 正文用 `null`。只有 `device` 模式把 access token 放入 Bearer 并对其取摘要。激活和刷新正文分别含激活密钥和刷新令牌。
4. 请求附上 `X-XPI-Challenge: <challenge_id>` 和 `X-XPI-Signature: <base64url 原始签名>`。请求仅发往可信 origin，拒绝重定向。设备级访问令牌须与证明的密钥配对。

```
JSON.stringify([
  "xpi-proof-v1", audience, purpose, challenge_id, nonce,
  method, pathIncludingQuery,
  sha256Hex(JCS(body ?? null)), sha256Hex(accessToken ?? "")
])
```

激活使用 `POST /api/v1/activations` 的 `{activation_key,device_name}`。续期使用 `POST /api/v1/tokens/refresh` 的 `{refresh_token}`。接收新会话凭据后由宿主更新持久状态。设备撤销、授权失效或挑战重放时应检查服务器错误并按宿主流程重新认证。

固定云端巡检

查询公开能力发现（`/api/v1/tianshu/capabilities`）确认巡检开放。云端工作进程执行运营方拥有的 `xiaosi-api-watch` 的官方稳定版本，范围限设备授权和运营方为账号登记的目标。请求输入是目标编号，使用可见版本与返回的工具 schema。URL、HTTP 方法、请求头和远程代码由服务端控制。

方法与路径（相对 `/api/v1`）	用途
`GET /cloud/plugins/xiaosi-api-watch/tools`	返回可用精确版本、`check_registered_endpoint` 的 `input_schema` 和本账号的 `endpoint_ids`。
`POST /cloud/executions`	创建任务。排队或执行中返回 202，同幂等标识的终态返回 200。
`GET /cloud/executions`	分页列出本账号任务。用 `state` 筛选，`limit` 默认为 20、上限 100，使用返回的 `next_cursor` 翻页。
`GET /cloud/executions/:id`	获取本账号任务状态与结果。
`DELETE /cloud/executions/:id`	取消本账号的排队或运行任务，已结束的任务返回现有终态。

状态包括 `queued`、`running`、`succeeded`、`failed`、`cancelled`。同一逻辑操作在网络失败后使用原幂等标识、版本与目标，签署请求时使用新挑战。新操作生成新幂等标识。取消不会回收已经送达目标站的 HEAD 请求。

```
{
  "plugin_id": "xiaosi-api-watch",
  "tool": "check_registered_endpoint",
  "version": "1.0.0",
  "input": { "endpoint_id": "ops_health" },
  "idempotency_key": "f22073af-e515-43d8-92b3-abd169c4024a"
}
```

启用云端适配的宿主还提供本机入口 `GET /plugins/xpi/cloud/tools`、`GET/POST /plugins/xpi/cloud/executions` 与 `GET/DELETE /plugins/xpi/cloud/executions/:id`。这些是使用本机宿主 Bearer 的回环接口，由宿主生成 XPI 设备证明。提交前向用户展示目标编号、版本和数据用途，确认后在上述 JSON 中额外提供 `confirmed: true`。本机确认字段由宿主处理。

运行限制和数据

- 单账号最多 3 个待执行任务、滚动 24 小时内 30 次创建。全站最多 50 个待执行、同一窗口 2,000 次创建和 5,000 条任务记录。工作进程逐项执行，排队超过 60 秒失效，单次连接与 DNS 合计最多 6 秒。
- 创建任务的请求体上限为 4 KiB。任务只对已登记的公网 HTTPS 目标发一次 HEAD，解析公网 IPv4 并验证原域名的 TLS。收到重定向时只记录响应状态。响应成功时返回 `http_status`、`ok` 与 `latency_ms`，其中 `ok` 表示 HTTP 2xx。
- 任务完成后保留 24 小时。结果仅含状态、耗时和固定错误码，任务记录按账号隔离。巡检目标可以记录收到的 HEAD 请求。平台发送的请求不携带用户凭据。
- `CLOUD_DISABLED` 表示当前环境未开通。遇到 `CLOUD_BUSY` 或 `RATE_LIMITED` 按响应退避。已运行过的任务在工作进程中断时可能进入失败终态。需要发起新的检查时使用新幂等标识。